Experiment No. : 9

Title: Analyze, Compare and Present Insights from a Real-World AI Use Case using Web Scraping and APIs

Batch: b3 **Roll No.: 16014225801** **Experiment No.: 9**

Aim

To acquire data using web scraping and/or APIs for a real-world AI use case, analyze the collected data, compare relevant observations, and present meaningful insights and recommendations.

Resources Required

- Python 3.x
- Selenium
- VS Code / PyCharm / Jupyter Notebook
- Web browser – Chrome/Edge/Firefox

Theory:

A real-world AI application requires relevant data to identify patterns, generate predictions, or support decision-making. Web scraping and APIs provide mechanisms for acquiring such data from online sources.
After collection, the data can be cleaned, analyzed, compared, and visualized to identify meaningful patterns.

Data Acquisition

Data can be collected using:

- Web scraping: Extracting relevant information from permitted website pages.
- APIs: Retrieving structured data through HTTP requests, commonly in JSON format.

Data Analysis

Data analysis helps identify:

- Differences between observations
- Trends over time
- Relationships between variables
- Unusual or unexpected values
- Findings relevant to the selected AI application

Data Visualization

Charts and tables make patterns easier to interpret.
Examples include:

- Line charts for trends over time
- Bar charts for comparisons
- Tables for presenting key observations

Role of AI

In this experiment, students analyze data relevant to an AI application and explain how the findings could support that application. Building or training an AI model is not mandatory unless specifically assigned by the instructor.

Suggested AI Use Cases

Select one of the following four use cases already discussed in the course.

Sr. No.	AI Use Case	Example Data to Acquire	Possible Analysis
1	Climate Intelligence for Heatwave Monitoring, Prediction and Early Warning	Temperature, humidity, rainfall and historical weather observations	Compare temperature patterns and identify periods of elevated heat risk

Important: Use legitimate, accessible data sources. For the urine-strip use case, prefer anonymized public research data or simulated measurements; do not collect identifiable patient information. For PocketQube, simulated data is acceptable when real telemetry is unavailable.

Expected Outcome

Students should produce a small, documented analysis of a real-world AI use case using data acquired through web scraping or APIs. The submission should include the source, code, cleaned dataset where appropriate, analysis, visualizations, insights, recommendations and limitations.

Result

Thus, data acquired using web scraping and/or APIs was analyzed and compared for a selected real-world AI use case. Relevant patterns and insights were presented through suitable visualizations, along with recommendations and limitations.

Code:

```
# Experiment No. 9
# Title: Analyze, Compare and Present Insights from a
# Real-World AI Use Case using Web Scraping and APIs
#
# Use Case: Climate Intelligence for Heatwave Monitoring,
# Prediction and Early Warning

import requests
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# ------------------------------------------------
# 1. DATA ACQUISITION USING API
# ------------------------------------------------

print("EXPERIMENT 9: CLIMATE INTELLIGENCE")
print("-" * 50)

# Mumbai coordinates
latitude = 19.0760
longitude = 72.8777
```

```
url = "https://archive-api.open-meteo.com/v1/archive"

params = {
    "latitude": latitude,
    "longitude": longitude,
    "start_date": "2025-01-01",
    "end_date": "2025-12-31",
    "daily": [
        "temperature_2m_max",
        "temperature_2m_min",
        "temperature_2m_mean",
        "relative_humidity_2m_mean",
        "precipitation_sum"
    ],
    "timezone": "Asia/Kolkata"
}

response = requests.get(url, params=params, timeout=30)
response.raise_for_status()

weather_data = response.json()

# Convert API response into a DataFrame
df = pd.DataFrame(weather_data["daily"])

df.rename(columns={
    "time": "Date",
    "temperature_2m_max": "Max_Temperature",
    "temperature_2m_min": "Min_Temperature",
    "temperature_2m_mean": "Mean_Temperature",
    "relative_humidity_2m_mean": "Humidity",
    "precipitation_sum": "Rainfall"
}, inplace=True)

df["Date"] = pd.to_datetime(df["Date"])

print("\nFirst five records:")
print(df.head())

print("\nDataset shape:", df.shape)

# ------------------------------------------------
# 2. DATA CLEANING
# ------------------------------------------------

print("\nMissing values before cleaning:")
print(df.isnull().sum())
```

```
df.drop_duplicates(subset="Date", inplace=True)

numeric_columns = [
    "Max_Temperature",
    "Min_Temperature",
    "Mean_Temperature",
    "Humidity",
    "Rainfall"
]

df[numeric_columns] = df[numeric_columns].apply(
    pd.to_numeric, errors="coerce"
)

df.dropna(subset=numeric_columns, inplace=True)

# Remove physically implausible observations
df = df[
    df["Min_Temperature"].between(-10, 50)
    & df["Max_Temperature"].between(-10, 60)
    & df["Humidity"].between(0, 100)
    & df["Rainfall"].ge(0)
].copy()

# ------------------------------------------------
# 3. DATA ANALYSIS
# ------------------------------------------------

df["Month"] = df["Date"].dt.month

monthly = df.groupby("Month").agg(
    Average_Max_Temperature=("Max_Temperature", "mean"),
    Average_Humidity=("Humidity", "mean"),
    Total_Rainfall=("Rainfall", "sum")
).reset_index()

print("\nMonthly Weather Analysis:")
print(monthly.round(2).to_string(index=False))

print("\nOverall Statistical Summary:")
print(df[numeric_columns].describe().round(2))

# ------------------------------------------------
# 4. HEAT-RISK INDICATOR
# ------------------------------------------------

# Illustrative screening rule, NOT an official heatwave
```

```
# definition or a validated health-risk prediction model.

df["High_Heat_Day"] = (
    df["Max_Temperature"] >= 35
).astype(int)

# Identify consecutive days with maximum temperature >= 35°C
df["Heat_Streak_Group"] = (
    df["High_Heat_Day"]
    .ne(df["High_Heat_Day"].shift())
    .cumsum()
)

df["Consecutive_Hot_Days"] = (
    df["High_Heat_Day"]
    * df.groupby("Heat_Streak_Group").cumcount().add(1)
)

df["Heat_Risk"] = "Low"

df.loc[
    (df["Max_Temperature"] >= 35)
    & (df["Humidity"] >= 60),
    "Heat_Risk"
] = "Elevated"

df.loc[
    df["Max_Temperature"] >= 38,
    "Heat_Risk"
] = "High"

print("\nHeat-risk category counts:")
print(df["Heat_Risk"].value_counts())

# ------------------------------------------------
# 5. IDENTIFY IMPORTANT OBSERVATIONS
# ------------------------------------------------

hottest_day = df.loc[df["Max_Temperature"].idxmax()]
coolest_day = df.loc[df["Min_Temperature"].idxmin()]
wettest_day = df.loc[df["Rainfall"].idxmax()]

print("\nKEY INSIGHTS")
print("-" * 50)

print(
    f"Hottest day: {hottest_day['Date'].date()} "
    f"({hottest_day['Max_Temperature']:.1f} °C)"
```

```
)

print(
    f"Coolest minimum-temperature day: "
    f"{coolest_day['Date'].date()} "
    f"({coolest_day['Min_Temperature']:.1f} °C)"
)

print(
    f"Wettest day: {wettest_day['Date'].date()} "
    f"({wettest_day['Rainfall']:.1f} mm)"
)

print(
    f"Days with maximum temperature >= 35°C: "
    f"{int(df['High_Heat_Day'].sum())}"
)

print(
    f"Days classified as elevated or high heat risk: "
    f"{df['Heat_Risk'].isin(['Elevated', 'High']).sum()}"
)

# ------------------------------------------------
# 6. DATA VISUALIZATION
# ------------------------------------------------

sns.set_theme(style="whitegrid")

# Graph 1: Daily maximum temperature
plt.figure(figsize=(12, 5))
plt.plot(
    df["Date"], df["Max_Temperature"],
    label="Maximum Temperature"
)
plt.axhline(
    35, linestyle="--",
    label="Illustrative 35°C threshold"
)
plt.title("Mumbai: Daily Maximum Temperature (2025)")
plt.xlabel("Date")
plt.ylabel("Temperature (°C)")
plt.legend()
plt.tight_layout()
plt.savefig("temperature_trend.png", dpi=150)
plt.show()

# Graph 2: Monthly temperature comparison
```

```
plt.figure(figsize=(10, 5))
sns.barplot(
    data=monthly,
    x="Month",
    y="Average_Max_Temperature"
)
plt.title("Average Maximum Temperature by Month")
plt.xlabel("Month")
plt.ylabel("Temperature (°C)")
plt.tight_layout()
plt.savefig("monthly_temperature.png", dpi=150)
plt.show()

# Graph 3: Humidity versus maximum temperature
plt.figure(figsize=(9, 5))
sns.scatterplot(
    data=df,
    x="Humidity",
    y="Max_Temperature",
    hue="Heat_Risk"
)
plt.title("Temperature vs Humidity")
plt.xlabel("Mean Relative Humidity (%)")
plt.ylabel("Maximum Temperature (°C)")
plt.tight_layout()
plt.savefig("temperature_humidity.png", dpi=150)
plt.show()

# Graph 4: Monthly rainfall
plt.figure(figsize=(10, 5))
sns.barplot(
    data=monthly,
    x="Month",
    y="Total_Rainfall"
)
plt.title("Total Monthly Rainfall")
plt.xlabel("Month")
plt.ylabel("Rainfall (mm)")
plt.tight_layout()
plt.savefig("monthly_rainfall.png", dpi=150)
plt.show()

# ------------------------------------------------
# 7. SAVE CLEANED DATASET AND ANALYSIS
# ------------------------------------------------

df.to_csv("mumbai_weather_cleaned.csv", index=False)
monthly.to_csv("monthly_weather_analysis.csv", index=False)
```

```
print("\nFiles generated successfully:")
print("1. mumbai_weather_cleaned.csv")
print("2. monthly_weather_analysis.csv")
print("3. temperature_trend.png")
print("4. monthly_temperature.png")
print("5. temperature_humidity.png")
print("6. monthly_rainfall.png")
```

Output:

```
EXPERIMENT 9: CLIMATE INTELLIGENCE
--------------------------------------------------

First five records:
        Date  Max_Temperature  Min_Temperature  Mean_Temperature  Humidity  \
0 2025-01-01             32.4             22.3              26.8        61
1 2025-01-02             32.8             21.1              26.7        58
2 2025-01-03             33.4             21.1              26.4        52
3 2025-01-04             33.1             19.1              25.3        48
4 2025-01-05             29.3             21.2              24.4        62

   Rainfall
0       0.0
1       0.0
2       0.0
3       0.0
4       0.0

Dataset shape: (365, 6)

Missing values before cleaning:
Date                0
Max_Temperature     0
Min_Temperature     0
Mean_Temperature    0
Humidity            0
Rainfall            0
dtype: int64

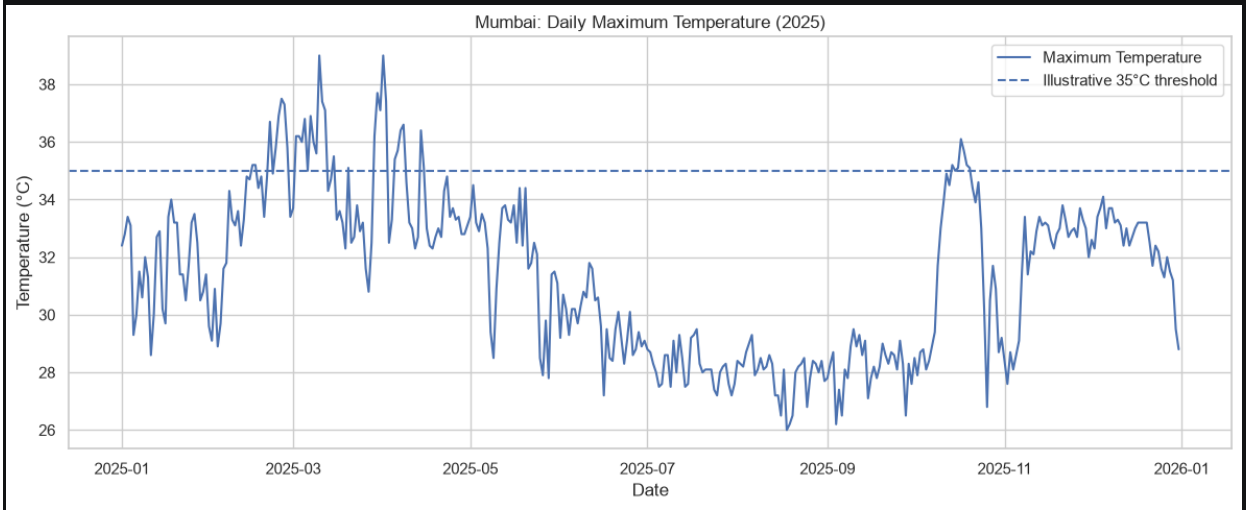
Monthly Weather Analysis:
 Month  Average_Max_Temperature  Average_Humidity  Total_Rainfall
     1                    31.64             55.19             0.0
     2                    33.84             42.68             0.0
     3                    34.80             50.45             0.0
     4                    34.04             68.27             1.7
     5                    31.99             76.61           184.2
     6                    29.67             84.63           703.8
```

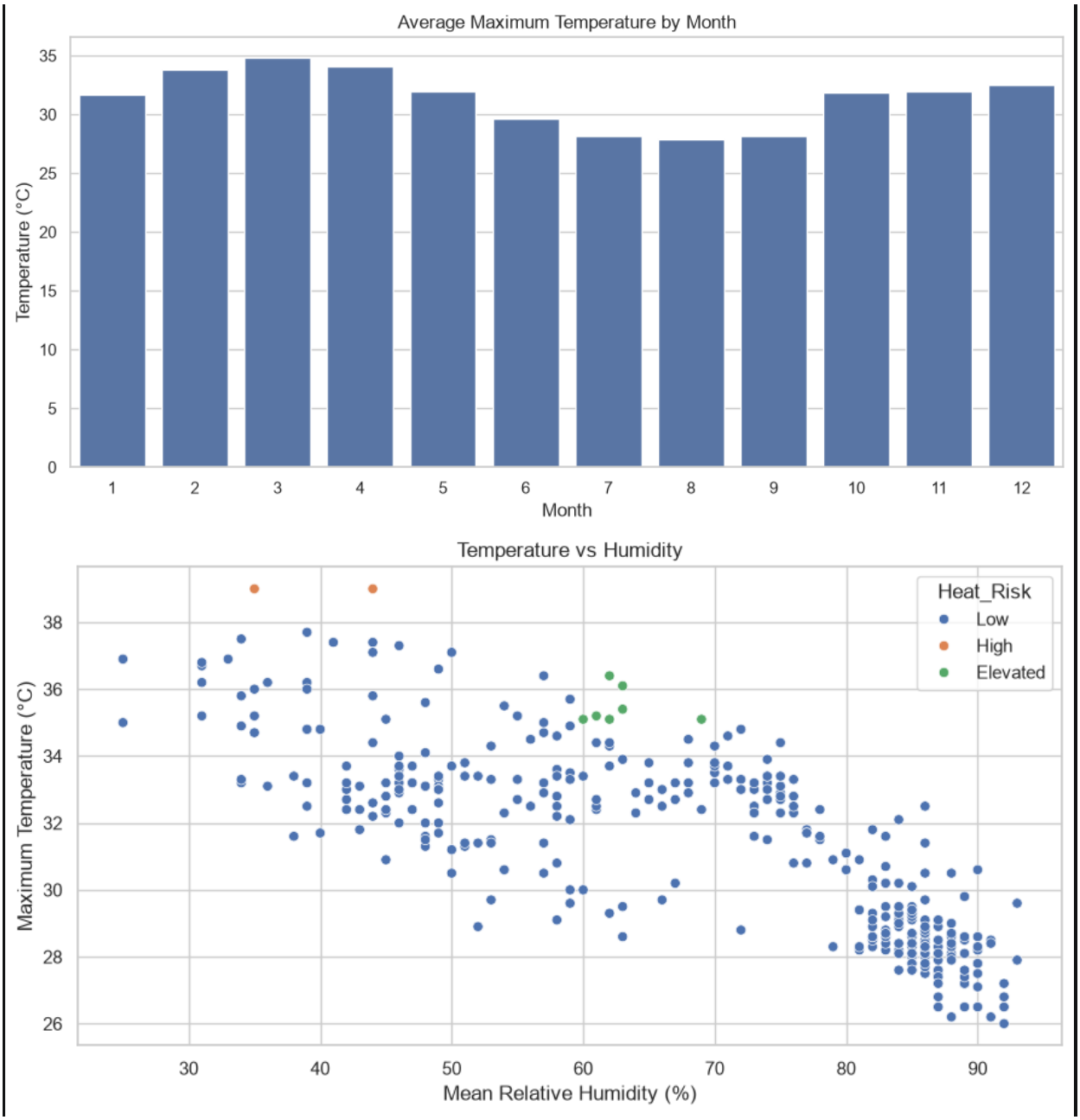
```
     7                  28.18            86.00          514.7
     8                  27.93            86.16          641.6
     9                  28.18            86.50          466.8
    10                  31.87            74.10          142.9
    11                  31.93            60.63           12.1
    12                  32.49            46.10            0.0

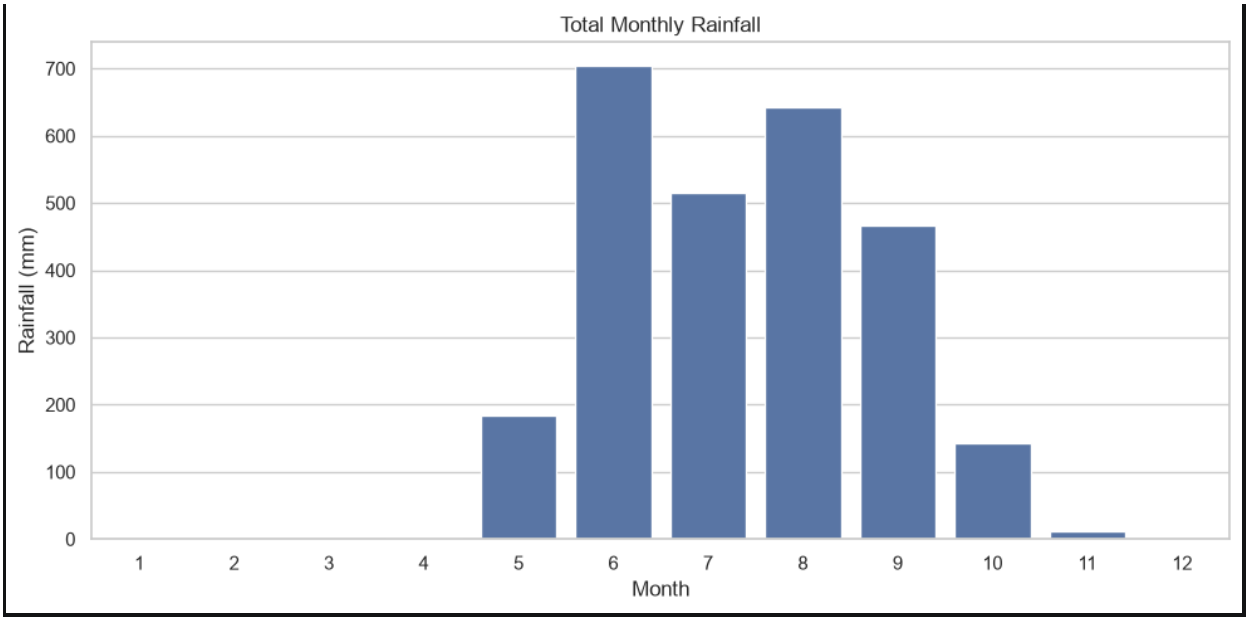
Overall Statistical Summary:
       Max_Temperature  Min_Temperature  Mean_Temperature  Humidity  Rainfall
count           365.00           365.00            365.00    365.00    365.00
mean             31.36            23.94             27.32     68.24      7.31
std               2.83             2.28              1.71     18.11     17.03
min              26.00            18.10             23.20     25.00      0.00
25%              28.60            22.40             26.20     52.00      0.00
50%              31.60            24.60             26.90     73.00      0.10
75%              33.30            25.50             28.80     85.00      7.40
max              39.00            28.80             31.80     93.00    134.90

Heat-risk category counts:
Heat_Risk
Low         356
Elevated      7
High          2
Name: count, dtype: int64

KEY INSIGHTS
--------------------------------------------------
Hottest day: 2025-03-10 (39.0 °C)
Coolest minimum-temperature day: 2025-12-31 (18.1 °C)
Wettest day: 2025-08-19 (134.9 mm)
Days with maximum temperature >= 35°C: 39
Days classified as elevated or high heat risk: 9
```




Average Maximum Temperature by Month
Temperature (°C)
Month
0
5
10
15
20
25
30
35
1
2
3
4
5
6
7
8
9
10
11
12
Temperature vs Humidity
Maximum Temperature (°C)
Mean Relative Humidity (%)
Heat_Risk
Low
High
Elevated
26
28
30
32
34
36
38
30
40
50
60
70
80
90




```
Files generated successfully:
1. mumbai_weather_cleaned.csv
2. monthly_weather_analysis.csv
3. temperature_trend.png
4. monthly_temperature.png
5. temperature_humidity.png
6. monthly_rainfall.png
```